

Lab – 5

Taxi Route Optimization using Q-Learning

[Tadi Akhil]^[DSC25027], [Tadi Dheeraj]^[DSC25028]

Objective:

The objective of this assignment is to explore an application-oriented Reinforcement Learning (RL) problem, formulate it using the standard RL components (agent, environment, state, action, reward and terminal condition), implement a suitable RL algorithm, and analyse the performance of the trained agent.

In this the Taxi Route Optimization problem is solved with the tabular Q-Learning algorithm. The agent learns, purely by trial and error, how to navigate a grid city, pick up a passenger and drop them at the requested destination in as few steps as possible while avoiding illegal actions.

Application Description:

Application / Domain:

Taxi Route Optimization belongs to the domain of intelligent transportation and logistics. Ride-hailing and delivery systems must continually decide how a vehicle should move to serve a request quickly and cheaply. Here the problem is modelled by the classic Taxi environment (Dietterich, 2000) provided by the Gymnasium library: a taxi moves on a 5×5 grid containing walls, and four designated locations – Red (R), Green (G), Yellow (Y) and Blue (B).

Problem to be Solved:

At the start of each episode the taxi is placed at a random square, a passenger is waiting at one of the four stations, and a destination station is chosen randomly. The taxi must (i) drive to the passenger, (ii) pick the passenger up, (iii) drive to the destination, and (iv) drop the passenger off. Walls block movement, and picking up or dropping off at the wrong place is penalised.

Objective of the RL Agent:

The agent must learn a policy $\pi(s)$ that maximises the expected cumulative (discounted) reward. In this environment that is equivalent to completing every trip with the shortest possible route and with zero illegal pick-up/drop-off attempts. A successful trip earns +20, each step costs -1, so a trip that takes N steps returns $20 - (N - 1) = 21 - N$.

RL Problem Formulation:

The task is modelled as a finite, episodic Markov Decision Process (MDP) $\langle S, A, P, R, \gamma \rangle$. Table 1 summarises each RL component.

Component	Description
Agent	The taxi driver (Q-Learning agent) that observes the state, chooses a movement or passenger action, and learns from the reward it receives.
Environment	Gymnasium Taxi environment: a 5×5 grid with internal walls and four stations (R, G, Y, B). The environment is deterministic; it applies the action, returns the new state, the reward and the termination flag.
State space	Discrete, $ S = 500$ states, encoded as an integer 0–499 from the tuple (taxi row, taxi column, passenger location, destination): • taxi row × column: $5 \times 5 = 25$ positions • passenger location: 5 values (R, G, Y, B, or inside the taxi) • destination: 4 values (R, G, Y, B) $25 \times 5 \times 4 = 500$ (of which 400 are reachable at the start of an episode)
Action space	Discrete, $ A = 6$: 0 – move South 1 – move North 2 – move East 3 – move West 4 – Pick up passenger 5 – Drop off passenger
Reward function	–1 for every time-step (encourages short routes) +20 for a successful drop-off at the correct destination –10 for an illegal “Pick-up” or “Drop-off” action
Terminal condition	Termination: the passenger is dropped off at the correct destination (success). Truncation: the episode is cut off after 200 steps (time limit) – counted as a failure.
Discount factor	$\gamma = 0.99$

Table 1: RL formulation of the Taxi Route Optimization problem

Why this is a good RL problem: The reward is delayed (+20 only at the very end of a trip), the agent must trade off exploration and exploitation, and the optimal behaviour (shortest legal route around walls) is not given to the agent – it has to be discovered from experience.

Algorithm Used: Q-Learning:

Working Principle:

Q-Learning (Watkins, 1989) is a model-free, off-policy temporal-difference control algorithm. It learns an action-value function $Q(s, a)$ – the expected discounted return of taking action a in state s and behaving optimally afterwards – and stores it in a table of size $|S| \times |A|$ (here 500×6). After every transition (s, a, r, s') the table is updated with the rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

The term in square brackets is the temporal-difference (TD) error: the difference between the bootstrapped target $r + \gamma \cdot \max Q(s', a')$ and the current estimate. Because the target uses the maximum over next actions rather than the action actually taken, Q-Learning learns the optimal policy independently of the exploratory behaviour policy (off-policy). When a state is terminal the target is simply r .

Behaviour policy: ϵ -greedy. With probability ϵ the agent picks a random action (exploration); otherwise it picks $\text{argmax}_a Q(s, a)$ (exploitation). ϵ starts at 1.0 and decays by a factor of 0.999 after each episode down to 0.01, so the agent explores widely early on and exploits its knowledge later.

Why Q-Learning Is Suitable:

- **Discrete and small state/action spaces:** $500 \times 6 = 3,000$ Q-values fit in a simple table, so no function approximation is needed (DQN would be unnecessarily complex).
- **Model-free:** the agent needs no knowledge of the grid, walls or reward rules – it learns directly from interaction.
- **Off-policy learning of the optimal route:** Q-Learning directly estimates the optimal action values, so the greedy policy converges to the shortest path. SARSA, being on-policy, would also learn about the exploratory moves and typically converges to a slightly more conservative policy.
- **Convergence guarantee and interpretability:** in the tabular case Q-Learning converges to Q^* if every state-action pair is visited sufficiently often, and the learned table can be inspected directly.

Training Parameters:

Parameter	Value	Role
Learning rate (α)	0.1	Step size of the TD update; small value gives stable learning.
Discount factor (γ)	0.99	Weights future rewards; high value because the +20 reward is delayed.
Initial ϵ	1.0	Fully random behaviour at the start (maximum exploration).
Minimum ϵ	0.01	Keeps 1% exploration at the end of training.
ϵ decay	0.999 / episode	Multiplicative decay; ϵ reaches 0.01 after $\approx 4,600$ episodes.
Training episodes	10,000	Total number of episodes used for learning.
Max steps / episode	200	Time limit (truncation) of the environment.
Q-table initialisation	All zeros	Neutral start; no prior knowledge.
Random seed	42	Makes the experiment reproducible.

Table 2: Hyper-parameters used for training

Implementation:

The solution is implemented in Python 3 using Gymnasium (environment), NumPy (Q-table) and Matplotlib (plots). The project contains two scripts: `taxi_qlearning.py` (training, evaluation, plots and screenshots) and `demo.py` (loads the trained Q-table and replays the agent). The core of the implementation is shown below; the full source code is submitted separately.

Q-Learning Agent:

```
class QLearningAgent:
    def __init__(self, n_states, n_actions, alpha, gamma, eps, eps_min, eps_decay):
        self.q = np.zeros((n_states, n_actions)) # Q-table (500 x 6)
        self.alpha, self.gamma = alpha, gamma
        self.eps, self.eps_min, self.eps_decay = eps, eps_min, eps_decay

    def act(self, state, greedy=False): # epsilon-greedy policy
        if not greedy and rng.random() < self.eps:
            return int(rng.integers(self.n_actions)) # explore
        best = np.flatnonzero(self.q[state] == self.q[state].max())
        return int(rng.choice(best)) # exploit (random tie-break)

    def update(self, s, a, r, s_next, terminated): # Q-learning TD update
        target = r if terminated else r + self.gamma * self.q[s_next].max()
        self.q[s, a] += self.alpha * (target - self.q[s, a])

    def decay_epsilon(self):
        self.eps = max(self.eps_min, self.eps * self.eps_decay)
```

Training Loop:

```
for ep in range(NUM_EPISODES):
    s, _ = env.reset(seed=SEED + ep)
    total, steps, success = 0.0, 0, 0
    while True:
        a = agent.act(s) # choose action
        s2, r, terminated, truncated, _ = env.step(a) # interact with environment
        agent.update(s, a, r, s2, terminated) # learn from the transition
        s, total, steps = s2, total + r, steps + 1
        if terminated: success = 1
        if terminated or truncated: break
    agent.decay_epsilon() # reduce exploration
    ep_reward[ep], ep_steps[ep], ep_success[ep] = total, steps, success
```

Design notes. Bootstrapping is switched off only on genuine termination and not on time-limit truncation, which is the theoretically correct treatment. Ties in the greedy choice are broken randomly so that an all-zero Q-table does not bias the agent towards action 0. After training, the greedy policy is evaluated on 1,000 episodes using seeds that were never used in training, and compared against a random-action baseline.

Training Details:

Item	Value
Environment	Gymnasium Taxi (v3; named v4 in Gymnasium ≥ 1.0 – identical dynamics)
Number of training episodes	10,000
Training time (CPU)	≈ 5 seconds
Average reward – first 100 episodes	-765.67
Average reward – last 100 episodes	7.62 (ϵ -greedy, $\epsilon = 0.01$)
Success rate – last 100 episodes	100.0 %
First episode with 100-ep. success rate ≥ 95 %	697
First episode with 100-ep. average reward > 0	2044
Final Q-table size	500×6 (saved as <code>q_table.npy</code>)

Table 3: Summary of the training run

```
=====
Q-LEARNING TRAINING - Taxi-v4
=====
Episodes=10000 alpha=0.1 gamma=0.99 eps: 1.0->0.01 (decay 0.999) max_steps=200 seed=42
|S| = 500 |A| = 6
=====
```

Episode	Avg Reward(100)	Success%(100)	Avg Steps(100)	Epsilon
1	-776.00	0.0%	200.0	1.000
500	-375.23	63.0%	134.3	0.607
1000	-66.73	100.0%	42.1	0.368
1500	-12.91	100.0%	20.1	0.223
2000	-3.22	100.0%	16.8	0.135
2500	4.19	100.0%	14.2	0.082
3000	5.35	100.0%	13.6	0.050
3500	5.82	100.0%	13.5	0.030
4000	6.69	100.0%	13.6	0.018
4500	7.67	100.0%	13.0	0.011
5000	7.49	100.0%	13.4	0.010
5500	7.77	100.0%	13.1	0.010
6000	7.28	100.0%	13.2	0.010
6500	7.96	100.0%	12.9	0.010
7000	6.63	100.0%	13.7	0.010
7500	7.01	100.0%	13.4	0.010
8000	7.11	100.0%	13.7	0.010
8500	7.53	100.0%	13.1	0.010
9000	7.83	100.0%	13.0	0.010
9500	7.32	100.0%	13.3	0.010
10000	7.62	100.0%	13.0	0.010

```
=====
Training finished in 5.0 s
=====
```

Figure 1: Console output during training (progress logged every 500 episodes)

Results and Learning Curve:

Learning Curve (Reward vs. Episodes):

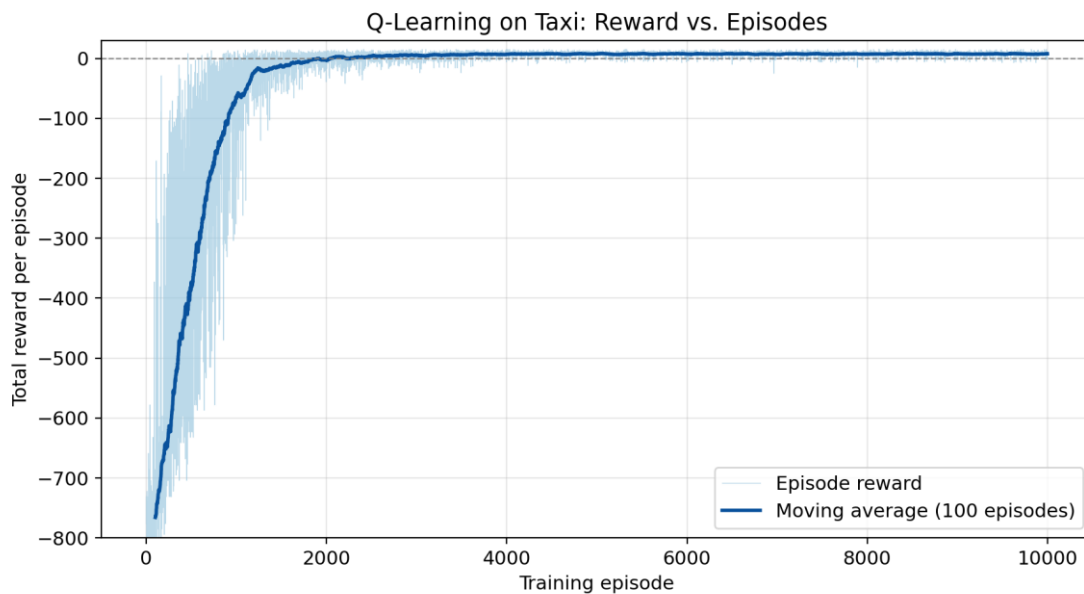


Figure 2: Reward vs. Episodes. Light blue: reward of each episode; dark blue: 100-episode moving average

The agent starts with a reward of about -770 per episode: acting randomly, it wanders for the full 200 steps and repeatedly performs illegal pick-up/drop-off actions (-10 each). The moving average then rises steeply between roughly episodes 200 and 1,500, crosses zero near episode 2,000, and flattens at about $+7.6$ from episode $\approx 3,000$ onwards.

Success Rate, Episode Length and Exploration:

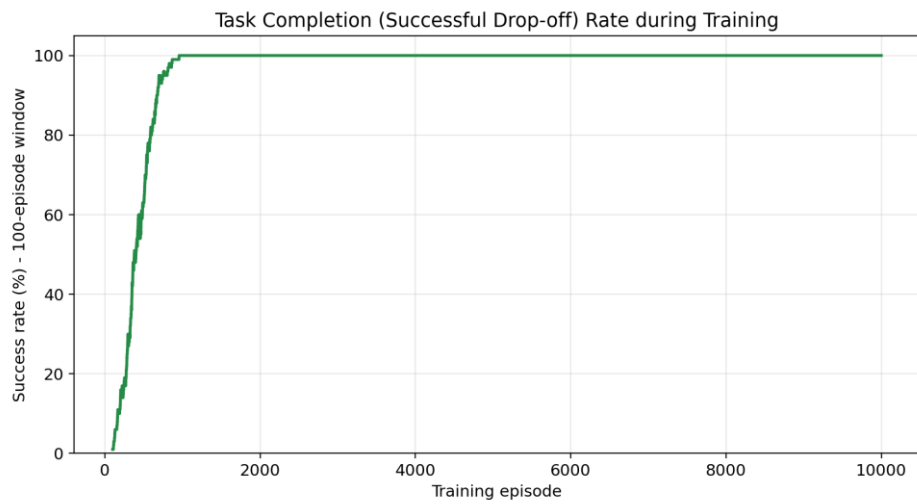


Figure 3: Success (task-completion) rate over training, 100-episode window

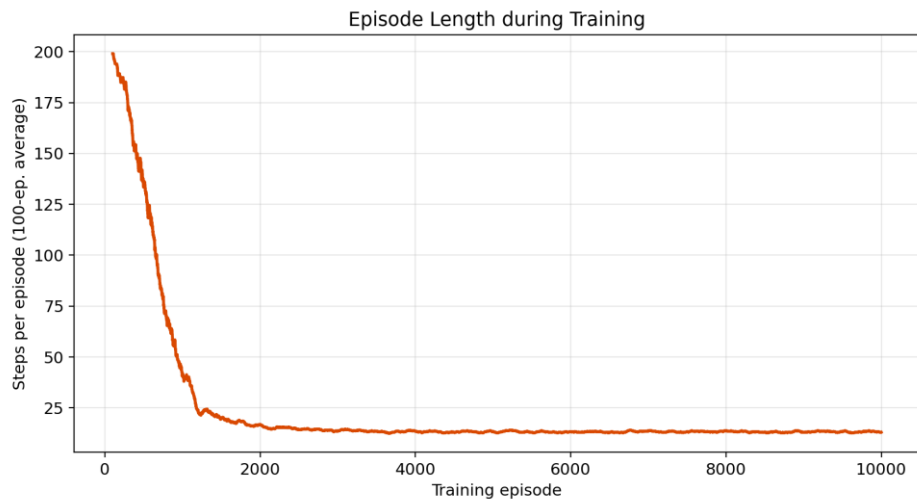


Figure 4: Average number of steps per episode (200 = time-out, ≈ 13 = near-optimal)

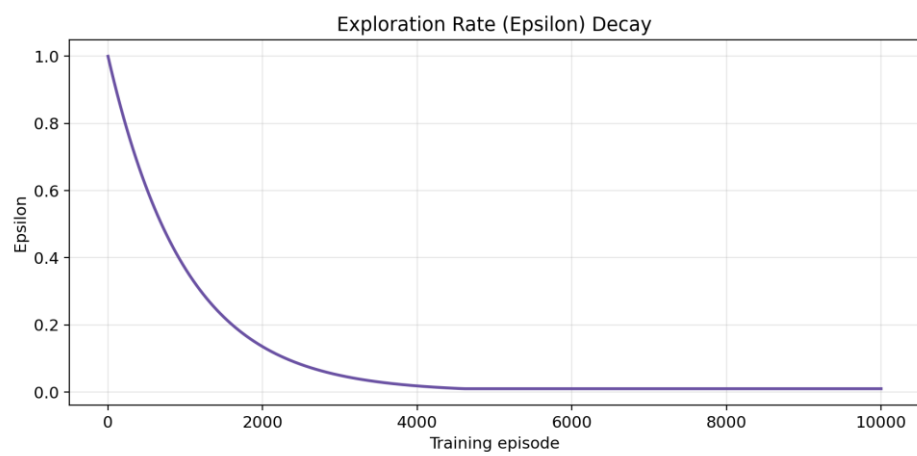


Figure 5: Exponential decay of the exploration rate ϵ

Performance in Training Phases:

Episodes	Avg. reward	Success rate (%)	Avg. steps / episode
1-2000	-189.07	81.3	71.53
2001-4000	4.81	100.0	14.06
4001-6000	7.37	100.0	13.23
6001-8000	7.33	100.0	13.30
8001-10000	7.41	100.0	13.23

Table 4: Training statistics in blocks of 2,000 episodes

Final Evaluation of the Trained Agent:

After training, the greedy policy ($\epsilon = 0$) was tested on 1,000 new episodes with seeds not used during training. A random agent was evaluated on the same seeds as a baseline.

Metric	Random agent	Trained Q-agent	Improvement
Average reward	-771.99	7.91	+779.90
Std. dev. of reward	104.08	2.57	–
Average steps per episode	196.32	13.09	–183.23 steps
Success rate (%)	4.5	100.0	+95.5 pp

Table 5: Greedy evaluation over 1,000 episodes – random baseline vs. trained agent

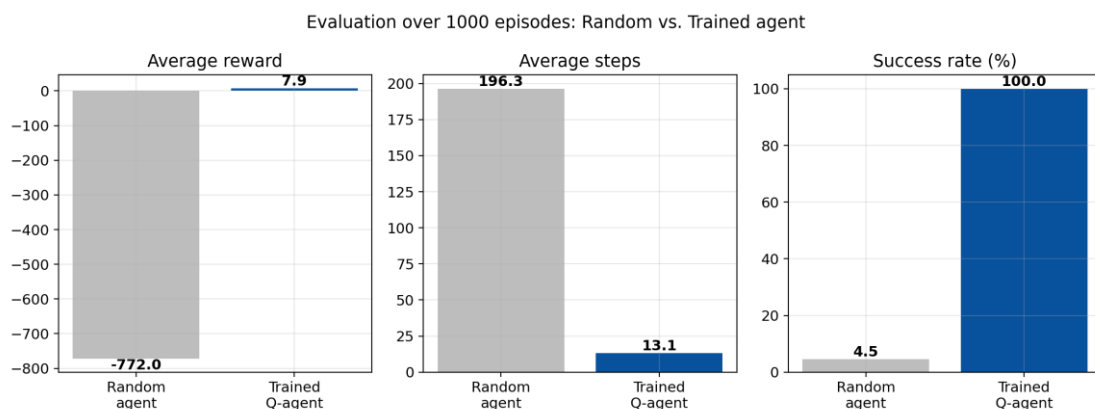


Figure 6: Evaluation comparison: random agent vs. trained Q-Learning agent

```
=====
FINAL RESULTS
=====
Training episodes           : 10000
Training time               : 5.0 s
First 100 episodes - avg reward : -765.67
Last 100 episodes - avg reward : 7.62 (epsilon-greedy)
Last 100 episodes - success rate : 100.0%
100-ep avg reward first > 0 at : episode 2044
100-ep success >= 95% first at : episode 697
-----
GREEDY EVALUATION (1000 unseen episodes)
  Trained agent: reward 7.91 +/- 2.57, steps 13.09, success 100.0%
  Random agent : reward -771.99 +/- 104.08, steps 196.32, success 4.5%
=====
Demonstration trip (seed 7) length : 11 steps
```

Figure 7: Console output of the final results

Application Screenshots:

Figure 8 shows one complete trip of the trained agent (greedy policy). The taxi (yellow car) starts right next to the passenger waiting at the Yellow station (bottom-left), picks the passenger up, drives up and across the grid around the walls, and drops the passenger at the Green station (top-right). The whole trip takes 11 steps, and returns a reward of 10, i.e. close to the maximum possible.

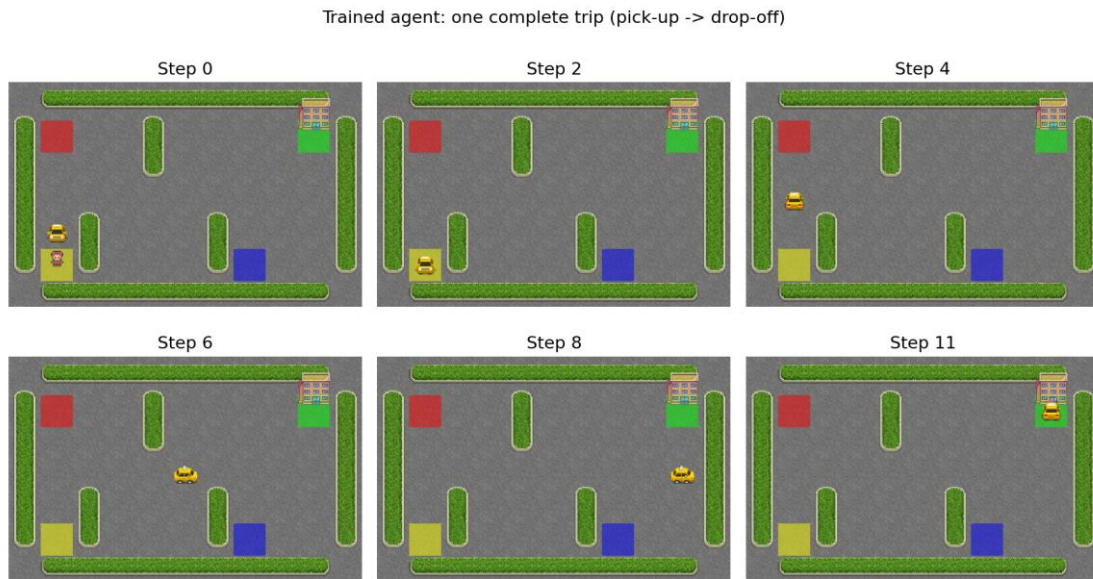


Figure 8: Frames from one complete trip of the trained agent (pick-up → drop-off)

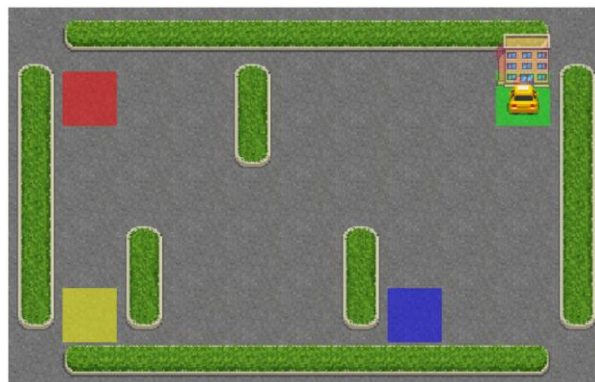


Figure 9: Final frame – passenger delivered at the destination

Performance Analysis:

Does Performance Improve as Training Progresses?:

Yes, clearly. The average reward rose from -765.67 (first 100 episodes) to 7.62 (last 100 episodes), the success rate rose from 0 % to 100 %, and the average episode length fell from the 200-step time-out to about 13 steps. The improvement happened in three recognisable stages:

1. **Exploration stage (episodes 1 – ≈ 700).** ϵ is high, the Q-table is mostly zeros and the agent behaves almost randomly. Rewards are around -770 to -300; the first successful trips are found by chance and the +20 reward begins propagating backwards through the table.
2. **Rapid-learning stage (≈ 700 – 2,500).** Success exceeds 95 % by episode ≈ 697. The reward signal has propagated to most states, so the agent follows increasingly direct routes; the moving-average reward turns positive at episode ≈ 2044. Remaining negative reward comes from detours and occasional random (exploratory) moves.

3. **Convergence stage ($\approx 3,000 - 10,000$).** The curve is flat at $\approx +7$ to $+8$ and the success rate is 100 %. Further training no longer improves the policy – the Q-values have converged.

Agent Performance:

- The trained greedy agent completed 100 % of 1,000 unseen test episodes, versus only 4.5 % for the random agent.
- Average test reward 7.91 with 13.09 steps per trip. Because a trip of N steps returns $21 - N$, the average trip length of ≈ 13 steps is consistent with the shortest-path distances on this grid, so the policy is essentially optimal.
- The test reward (7.91) is slightly higher than the last-100 training reward (7.62) because the training figure still includes 1 % random exploration moves, which occasionally add wasted steps.
- The standard deviation of 2.57 does not indicate unstable behaviour; it reflects the different initial taxi/passenger/destination distances of each episode.

Training Behaviour:

Reward and episode length are strongly correlated: while the agent is unskilled, most episodes end by time-out (200 steps) and accumulate large negative returns from illegal actions. The variance of the raw reward (light-blue band in Figure 2) is very high in the first $\approx 1,500$ episodes and then almost vanishes, which signals that the policy has stabilised. The ϵ schedule played an important role: ϵ decays from 1.0 to ≈ 0.05 within 3,000 episodes, so the plateau begins just as exploration becomes small.

Observations:

- The delayed +20 reward is initially discovered only by random exploration; once found, it propagates backwards through the Q-table one TD update at a time, which explains the S-shaped learning curve.
- The -10 penalty quickly teaches the agent to avoid illegal pick-up/drop-off actions; the -1 step cost then drives it toward the shortest route.
- A high discount factor ($\gamma = 0.99$) is appropriate here because the only positive reward is at the end of the trip.
- Decaying ϵ is essential: purely random behaviour never refines the policy, while too fast a decay would freeze the agent before it has found good routes.
- Training is very cheap (≈ 5 s for 10,000 episodes) thanks to the small state space.
- Limitations: (i) the result is from a single random seed – a multi-seed study would give confidence intervals; (ii) the environment is deterministic, so conclusions about stochastic settings (traffic, random demand) cannot be drawn; (iii) the Q-table grows exponentially with the number of state variables, so larger or continuous city maps would require function approximation such as DQN.

Possible extensions: compare against SARSA, tune α / ϵ -decay, repeat over several seeds, enlarge the map, or add stochastic traffic penalties.